Bash Scripting - One-In-All Handbook (Deep Guide)

Bash Scripting - One-In-All Handbook (Deep Guide)

This guide is a comprehensive reference and quick reminder for Bash scripting on Linux. It covers core concepts, variables, operators, conditionals, loops, arrays, user input, script arguments, redirection and pipes, exit status and error codes, debugging, file and directory handling, text processing, cron jobs, comments, functions, signals and traps, here-docs, quoting and expansion, portability, and more. All examples are plain-text and code, without tables or boxes, to keep it clean and printable.

Bash Scripting - One-In-All Handbook (Deep Guide)

1) What is a Bash script?

A Bash script is a plain text file containing commands executed by the Bash shell (/bin/bash). Scripts automate tasks such as backups, deployments, ETL jobs, checks, and system administration. To run a script: make it executable and execute it from the shell.

```
#!/usr/bin/env bash
echo "Hello from Bash"
ls -la
exit 0
```

Professional skeleton

```
#!/usr/bin/env bash
set -Eeuo pipefail
IFS=$'\n\t'

log(){ printf '[%s] %s\n' "$(date +%F\ %T)" "$*"; }
die(){ log "ERROR: $*"; exit 1; }
main(){
  log "Starting"
  # your logic here
}
main "$@"
```

2) How Bash runs commands and scripts

Bash reads a line, performs expansions (brace, tilde, parameter, command substitution, arithmetic, globbing), does word splitting, then executes the command. Path resolution uses PATH for non-absolute names. Sourcing (.) executes in current shell; running executes in a child shell process.

```
# Run in a new process
./myscript.sh
# Source into current shell
. ./myscript.sh   # or: source myscript.sh
```

Bash Scripting - One-In-All Handbook (Deep Guide)

3) Variables in Bash (deep)

3.1 Basics and assignment

Variables are untyped strings by default. No spaces around =. Use quotes to preserve spaces and avoid globbing.

```
name='Alice'
greeting="Hello $name"
echo "$greeting"
```

3.2 Numeric variables and arithmetic contexts

Use arithmetic expansions $((...)) or arithmetic command ((...)) for integer math. Optional declare -i enforces integer behavior on a variable.

```
a=10; b=3
sum=$(( a + b ))
((a+=5))
declare -i n=7; n+=2; echo "$n"
```

3.3 Arrays (indexed) meaning, definition, purpose, types

Indexed arrays are ordered lists accessed by numeric index starting at 0. Purpose: store lists such as filenames, IDs, or options. Type: 'indexed array'.

```
declare -a nums=(10 20 30)
echo "First: ${nums[0]}"
echo "All:   ${nums[@]}"
echo "Idxs:  ${!nums[@]}"
echo "Len:   ${#nums[@]}"
nums+=(40 50)
unset 'nums[1]'
```

3.4 Arrays (associative) meaning, definition, purpose, types

Associative arrays are key-value maps indexed by strings. Purpose: store settings, counters by name, or lookups. Enable with declare -A.

```
declare -A ages=([alice]=30 [bob]=28)
echo "Alice: ${ages[alice]}"
for k in "${!ages[@]}"; do echo "$k => ${ages[$k]}"; done
```

3.5 Environment variables, exported variables, and local

Exported variables become part of the environment of child processes. Local variables exist in current shell. Use local inside functions to limit scope.

```
export PATH="$HOME/bin:$PATH"
APP_HOME=/opt/app
myfunc(){ local tmp=42; echo "$tmp"; }
myfunc; echo "Global PATH: $PATH"
```

3.6 Positional and special parameters

```
echo "$0"      # script name
echo "$1"      # first argument
echo "$#"      # number of args
echo "$@"      # all args, split as words
echo "$*"      # all args as one word (joined by IFS)
```

```
echo "$?"      # exit status of last command
echo "$$"      # PID of current shell
echo "$!"      # PID of last background job
```

3.7 Quoting rules and best practices

Double quotes expand variables and command substitutions but prevent word splitting and globbing. Single quotes prevent all expansions. Always quote variables in tests and path operations.

```
f="/tmp/a b.txt"
rm "$f"   # safe
echo "$var"   # safe expansion
```

3.8 Parameter expansion techniques

```
v=${v:-default}
v=${v:=default}
: ${REQUIRED:?missing var}
len=${#v}
sub=${v:2:3}
x=${v/foo/bar}
x=${v//foo/bar}
x=${v#prefix}
x=${v%suffix}
```

4) Operators in Bash and why to use them

4.1 Arithmetic operators and contexts

```
(( a=5+3 ))   # + - * / % **
(( a+=2, b=a++ ))
result=$(( (a*b) % 7 ))
```

4.2 Numeric comparison: -eq vs = and (( ))

Use -eq, -ne, -lt, -le, -gt, -ge for numeric comparisons inside [ ] (test). Use = or != for string comparisons. In arithmetic context (( )), you can write a < b, a == b, etc. Reason: Bash treats values as strings unless you use numeric operators or arithmetic context. Using = compares text, not numbers, so 10 and 2 are just strings. Use -gt to compare them numerically.

```
a=10; b=2
[ "$a" -gt "$b" ] && echo "10 is greater"
[ "$a" = "$b" ] && echo "equal strings" || echo "different strings"
if (( a > b )); then echo "numeric compare via (( ))"; fi
```

4.3 String operators

```
[ "$s" = "hello" ]
[ "$s" != "world" ]
[ -n "$s" ]  # non empty
[ -z "$t" ]  # empty
[[ "$s" < "zoo" ]]  # lexicographic in [[ ]]
```

4.4 Logical operators

```
[ -f file ] && echo exists || echo missing
if [[ -d src && -x build.sh ]]; then ./build.sh; fi
! false && echo ok
```

4.5 File test operators

```
[ -e path ]  # exists
[ -f file ]  # regular file
[ -d dir  ]  # directory
[ -s file ]  # size > 0
[ -r file ]  # readable
[ -w file ]  # writable
[ -x file ]  # executable
[ -L link ]  # symlink
[ file1 -nt file2 ]  # newer than
[ file1 -ot file2 ]  # older than
```

4.6 Regex matching in [[ ]]

```
s="abc123"
if [[ $s =~ ^[a-z]+[0-9]+$ ]]; then echo match; fi
```

5) Conditional statements: purpose and usage

Conditionals choose code paths based on tests. Bash offers if/elif/else and case/esac.

5.1 if/elif/else

```
read -p "Age: " age
if (( age >= 18 )); then
  echo Adult
elif (( age >= 13 )); then
  echo Teen
else
  echo Child
fi
```

5.2 case/esac

```
read -p "Answer [y/n]: " a
case "$a" in
  y|Y|yes) echo Proceed;;
  n|N|no)  echo Abort;;
  *)       echo Invalid;;
esac
```

6) Loop statements: purpose, usage, types

6.1 for-in over a list

```
for name in Alice Bob Carol; do
  echo "Hello $name"
done
```

6.2 C-style for loop

```
for ((i=0; i<5; i++)); do
  echo "$i"
done
```

6.3 while loop

```
i=1
while (( i <= 3 )); do
  echo "$i"; ((i++))
done
```

6.4 until loop

```
x=0
until (( x == 3 )); do
  echo "$x"; ((x++))
done
```

6.5 break and continue

```
for i in {1..10}; do
  (( i==3 )) && continue
  (( i==6 )) && break
  echo "$i"
done
```

6.6 Reading files safely in loops

```
while IFS= read -r line; do
  printf '%s\n' "$line"
done < input.txt
```

6.7 select (menus)

```
PS3='Pick a fruit: '
select f in apple banana cherry; do
  echo "You picked $f"; break
done
```

7) User input: types, meaning, definition, defining user input

User input in Bash is primarily collected using read, select, and command line flags via getopts.

7.1 read: prompt and read a line

```
read -p "Enter name: " name
echo "Hello $name"
```

7.2 read with timeout, silent, arrays

```
read -t 5 -p "Answer within 5s: " ans || echo timeout
read -s -p "Password: " pass; echo
read -a items -p "Enter items: "  # fills an indexed array
```

7.3 select: menu input

```
select opt in start stop status; do
  echo "You chose: $opt"; break
done
```

7.4 getopts: parse -f style flags

```
while getopts ":u:p:v" opt; do
  case $opt in
    u) user=$OPTARG;;
    p) pass=$OPTARG;;
    v) verbose=1;;
    :) echo "Option -$OPTARG requires an argument"; exit 2;;
    \?) echo "Unknown option -$OPTARG"; exit 2;;
  esac
done
shift $((OPTIND-1))
```

8) Script arguments: meaning and usage

Arguments are passed after the script name and accessed via $1, $2, ..., $@, $#. Use shift to consume.

```
# myscript.sh file1 file2
echo "Script: $0"
echo "Count:  $#"
for a in "$@"; do echo "Arg: $a"; done
while [[ $# -gt 0 ]]; do
  echo "Processing: $1"; shift
done
```

9) Redirections and pipes

Redirect stdout, stderr, and use pipelines to chain commands.

```
cmd > out.txt        # stdout to file (truncate)
cmd >> out.txt       # append
cmd 2> err.txt       # stderr to file
cmd > out 2>&1       # both to out
cmd 2>&1 | tee log   # stream both into pipe
grep foo file | sort | uniq -c | sort -nr
```

Here-strings and here-docs

```
cat <<< "single line"
cat <<'EOF'
multi line text
EOF
```

10) Exit status and error codes

Every command returns an exit code in $?. 0 means success, non-zero indicates failure. Use set -e to exit on first error in simple scripts, or handle errors explicitly with || and if.

```
false; echo $?      # prints non-zero
true;  echo $?      # prints 0
cp src dst || { echo "copy failed"; exit 1; }
```

Custom exit codes

```
die(){ echo "ERROR: $*" >&2; exit 2; }
[[ -f "$1" ]] || die "Missing file: $1"
```

11) Debugging commands and techniques

Trace execution with set -x and PS4, validate with shellcheck, inspect variables and functions, use trap to capture errors and line numbers.

```
set -x            # enable xtrace
set +x            # disable xtrace
PS4='+${BASH_SOURCE}:${LINENO}:${FUNCNAME[0]}: '
set -x
trap 'echo "ERR at ${BASH_SOURCE}:${LINENO} (status=$?)"' ERR
```

Useful checks

```
type -a cmd        # where is a command resolved
declare -p var     # show attributes and value
compgen -c | wc -l # list commands available
```

12) File and directory handling

Create, list, find, copy, move, delete safely. Always quote paths.

```
mkdir -p "$dst"
cp -r "$src" "$dst"   # recursive copy
mv -- "$old" "$new"
rm -f -- "$file"
rm -rf -- "$dir"
```

find basics

```
find /var/log -type f -name '*.log' -size +10M -mtime +7 -print
find . -maxdepth 1 -type f -print0 | xargs -0 ls -l
```

Permissions and modes

```
chmod 654 file    # rw-r-xr--
chmod 000 file    # no permissions
chown user:group file
umask 022         # default mode mask
```

13) Text processing commands (quick remind)

grep

```
grep -n "pattern" file
grep -r "TODO" src/
```

sed

```
sed -n '1,10p' file
sed 's/foo/bar/g' file
sed -E 's/^([0-9]+),([a-z]+)/\2 \1/' data.csv
```

awk

```
awk -F, '{sum+=$3} END{print sum}' data.csv
awk '{print NR ":" $0}' file
```

cut, tr, sort, uniq, paste, join

```
cut -d, -f2 file
tr '[:lower:]' '[:upper:]' < file
sort file | uniq -c | sort -nr
paste f1 f2
join -1 1 -2 1 f1 f2
```

xargs and parallelism

```
printf '%s\n' file1 file2 | xargs -n1 -P4 gzip
```

14) Cron jobs: scheduling scripts

Use crontab -e to edit per-user crontab. Format: m h dom mon dow command. Use absolute paths and environment variables as needed.

```
crontab -e
# Every day at 02:30
30 2 * * * /home/user/backup.sh >> /home/user/backup.log 2>&1
# Every 5 minutes
*/5 * * * * /usr/local/bin/health-check
```

Systemd timers (alternative)

```
# On systemd systems consider timers for reliability
# Create .service and .timer units
```

15) Comments and coding style

Use # for comments. Keep scripts readable, modular, and safe. Quote variables, set -Eeuo pipefail, use functions, and return explicit exit codes.

```
# This script deploys the application
set -Eeuo pipefail
# TODO: add retries
```

16) Functions: definition and use

Functions group commands and can accept arguments via $1, $2 or named locals. Return values are exit codes or printed output captured with command substitution.

```
greet(){
  local who=${1:-world}
  echo "Hello, $who"
}
msg=$(greet Alice)
echo "$msg"
```

17) Signals and traps

Trap signals to cleanup temporary files or stop gracefully.

```
tmp=$(mktemp)
cleanup(){ rm -f "$tmp"; }
trap cleanup EXIT INT TERM
echo data > "$tmp"
sleep 2
echo done
```

18) Globbing and brace expansion

Globbing expands wildcards like *. Brace expansion generates strings before globbing.

```
echo src/*.c
echo file_{a,b,c}.txt
```

19) Subshells vs current shell; sourcing vs executing

Parentheses create a subshell; changes inside do not affect the parent shell. Sourcing runs in current shell.

```
(cd /tmp; echo "Here: $(pwd)")
echo "Back: $(pwd)"
. ./env.sh  # source into current shell
```

20) Portability notes: POSIX sh vs Bash

For portability, avoid Bash-only features (arrays, [[ ]], =() maps, process substitution) and target /bin/sh. If you need Bash features, use shebang #!/usr/bin/env bash and document Bash requirement.

21) Script templates you can reuse

21.1 Minimal strict-mode script

```
#!/usr/bin/env bash
set -Eeuo pipefail
IFS=$'\n\t'
main(){
  echo "Hello"
}
main "$@"
```

21.2 CLI script with getopts

```
#!/usr/bin/env bash
set -Eeuo pipefail
show_help(){ echo "Usage: $0 -i INPUT -o OUTPUT [-v]"; }
input= output= verbose=0
while getopts ":i:o:vh" opt; do
  case $opt in
    i) input=$OPTARG;;
    o) output=$OPTARG;;
    v) verbose=1;;
    h) show_help; exit 0;;
    :) echo "Missing arg for -$OPTARG"; exit 2;;
    \?) echo "Unknown -$OPTARG"; exit 2;;
  esac
done
shift $((OPTIND-1))
[[ -n "$input" && -n "$output" ]] || { show_help; exit 2; }
echo "Processing $input -> $output"
```

22) Quick reminder lines

Use [[ ]] for safer string tests; use (( )) for numeric logic. Quote paths. Prefer arrays over parsing ls. Use mapfile/readarray to read lines into arrays. Prefer grep -E for regex. Use pipefail to fail pipelines. Prefer printf over echo for portability. Avoid for i in $(cmd) because of word splitting; use while read -r. Check exit codes, log steps, and always test scripts with set -x in a non-production environment.

Bash Script: Definition and Types

Definition:

A Bash script is a text file containing a series of commands that are executed by the Bash shell to automate tasks or perform operations in Linux or Unix systems.

Structure:

```
#!/bin/bash
# This is a comment
echo "Hello, World!"
```

How to Run:

1. Create a file: nano myscript.sh
2. Add commands inside it.
3. Give permission: chmod +x myscript.sh
4. Run: ./myscript.sh

Types of Bash Scripts

---

1. Startup (Initialization) Scripts

- Run automatically when the system or shell starts.

Examples: /etc/profile, ~/.bashrc

2. System Administration Scripts

- Used by admins for automation (backups, updates, etc.).

Example:

```
tar -czf backup_$(date +%F).tar.gz /home/user/Documents
```

3. Automation Scripts

- Automate repetitive user tasks.

Example:

```
for file in *.txt; do mv "$file" processed_"$file"; done
```

4. User Interaction Scripts

- Take user input and give output.

Example:

```
read -p "Enter your name: " name
echo "Welcome, $name!"
```

5. Monitoring or Utility Scripts

- Monitor system resources or logs.

Example:

```
df -h > disk_usage.log
```

6. Network Scripts

- Handle or monitor network tasks.

Example:

```
ping -c 4 google.com > network_check.log
```

7. Conditional and Control Scripts

- Use logic, loops, and conditions.

Example:

```
if [ -f "/etc/passwd" ]; then
  echo "File exists!"
else
  echo "File not found!"
fi
```

Summary Table

| Type | Description | Example Use Case |

|-------|-------------|------------------|

| Startup Script | Runs at shell startup | .bashrc |

| System Admin Script | Automates maintenance | Backups |

| Automation Script | Repeats common tasks | File renaming |

| User Interaction Script | Takes input | Setup tools |

| Monitoring Script | Tracks system status | Disk usage |

| Network Script | Network monitoring | Ping tests |

| Control Script | Uses logic & loops | Conditional jobs |

Types of Variables in Bash

1. Local Variables

- Available only in the current shell or function.

Example:

```
name="Alice"
echo $name
```

2. Environment Variables

- Global variables available to child processes.

Example:

```
export USERNAME="bob"
bash -c 'echo $USERNAME'
```

3. Shell Variables

- Built-in variables defined by the shell.

Examples: $HOME, $PWD, $PATH, $USER, $SHELL, $RANDOM

4. Positional Parameters

- Represent arguments passed to a script.

Example:

```
echo "First argument: $1"
```

5. Special Variables

- Provide information about shell or scripts.

Examples:

```
$0 (script name)
$? (exit status)
$$ (PID)
$@ (all arguments)
```

6. Array Variables

- Store multiple values.

Example:

```
fruits=("apple" "banana" "cherry")
echo ${fruits[1]}
```

7. Read-only Variables

- Cannot be changed after assignment.

Example:

```
readonly pi=3.14
```

Defining Each Type of Operator in Bash

Definition:

Operators in Bash are special symbols or keywords used to perform operations such as arithmetic calculations, comparisons, logic, string testing, and file handling.

---

1. Arithmetic Operators

Used for mathematical calculations.

Syntax:

```
$(( expression ))
```

Example:

```
a=10
b=5
echo $((a + b))   # 15
echo $((a ** b))  # 100000
```

Operators:

+ Addition

- Subtraction

* Multiplication

/ Division

% Modulus

** Exponentiation

---

2. Relational (Comparison) Operators

Used to compare two numeric values.

Syntax:

```
[ expression ]
```

Example:

```
a=8
b=10
if [ $a -lt $b ]; then
  echo "a is less than b"
fi
```

Operators:

-eq Equal to

-ne Not equal to

-gt Greater than

-lt Less than

-ge Greater or equal

-le Less or equal

---

3. Boolean (Logical) Operators

Used to combine or invert conditions.

Syntax:

```
[[ condition1 && condition2 ]]
```

Example:

```
a=5
b=15
if [[ $a -lt 10 && $b -gt 10 ]]; then
  echo "Both conditions true"
```

```
fi
```

Operators:

```
!   NOT
&&  AND
||  OR
```

---

4. String Operators

Used to compare or test strings.

Example:

```
str1="hello"
str2="world"
if [ "$str1" != "$str2" ]; then
  echo "Strings differ"
fi
```

Operators:

```
=   Equal
!=  Not equal
-z  String empty
-n  String not empty
<   Less than alphabetically
>   Greater than alphabetically
```

---

5. File Test Operators

Used to check file existence, type, and permissions.

Example:

```
file="/etc/passwd"
if [ -f "$file" ]; then
  echo "File exists"
fi
```

Operators:

-e File exists

-f Regular file

-d Directory

-r Readable

-w Writable

-x Executable

-s Not empty

---

6. Assignment Operators

Used to assign or modify variable values.

Example:

```
a=5
((a+=3))  # a = 8
((a*=2))  # a = 16
```

Operators:

= Assign

+= Add and assign

-= Subtract and assign

*= Multiply and assign

/= Divide and assign

Summary Table

Type	Purpose	Example
Arithmetic	Math operations	$((a + b))
Relational	Number comparison	[$a -lt $b]
Logical	Combine conditions	[[$a -gt 0 && $b -lt 10]]
String	Compare text	["$a" = "$b"]
File Test	File checking	[-f /etc/passwd]
Assignment	Assign/update vars	((a+=1))

Operators in Bash

Definition:

Operators in Bash are special symbols or keywords used to perform operations on data such as arithmetic calculations, comparisons, logic, string tests, and file checks.

Types of Operators in Bash

---

1. Arithmetic Operators

Used for mathematical calculations.

Examples:

+ (Addition) $((a + b))

- (Subtraction) $((a - b))

* (Multiplication) $((a * b))

/ (Division) $((a / b))

% (Modulus) $((a % b))

** (Exponent) $((a ** b))

2. Relational (Comparison) Operators

Used to compare numeric values.

Examples:

-eq Equal to

-ne Not equal to

-gt Greater than

-lt Less than

-ge Greater than or equal to

-le Less than or equal to

3. Boolean (Logical) Operators

Used to combine or invert conditions.

Examples:

- ! NOT
- -a AND
- -o OR

Modern syntax:

```
[[ $a -lt 10 && $b -gt 5 ]]
```

4. String Operators

Used to compare strings.

Examples:

- = Equal
- != Not equal
- < Less than (alphabetically)
- > Greater than (alphabetically)
- -z String is empty
- -n String is not empty

5. File Test Operators

Used to test file types and properties.

Examples:

- -e file True if file exists
- -f file True if regular file
- -d file True if directory
- -r file True if readable
- -w file True if writable
- -x file True if executable
- -s file True if file not empty

6. Assignment Operators

Used to assign or update variable values.

Examples:

- = Assign

+= Add and assign

-= Subtract and assign

*= Multiply and assign

/= Divide and assign

Example:

```
#!/bin/bash
a=8
b=3
if [[ $a -gt $b && $b -ne 0 ]]; then
  echo "a is greater than b and b is not zero"
  echo "Sum: $((a + b))"
fi
```

Summary Table

Category	Examples	Used For
Arithmetic	+, -, *, /, %, **	Math operations
Relational	-eq, -lt, -gt	Compare numbers
Logical	&&, ||, !	Combine conditions
String	=, !=, -z, -n	Compare text
File Test	-f, -d, -e	File/dir checking
Assignment	=, +=, -=, *=, /=	Assign/update values

Conditional Statements in Bash

Definition:

Conditional statements in Bash allow you to make decisions in scripts, executing specific commands based on whether a condition is true or false.

Types of Conditional Statements in Bash

---

1. if Statement

Executes commands only if a condition is true.

Syntax:

```
if [ condition ]
then
    commands
fi
```

Example:

```
a=10
if [ $a -gt 5 ]; then
  echo "a is greater than 5"
fi
```

---

2. if-else Statement

Executes one block if true, another if false.

Syntax:

```
if [ condition ]
```

```
then
    commands_if_true
else
    commands_if_false
fi
```

Example:

```
num=8
if [ $num -gt 10 ]; then
  echo "Number greater than 10"
else
  echo "Number less than or equal to 10"
fi
```

---

3. if-elif-else Ladder

Used for multiple conditions.

Syntax:

```
if [ condition1 ]
then
    commands1
elif [ condition2 ]
then
    commands2
else
    commands3
fi
```

Example:

```
marks=75
if [ $marks -ge 90 ]; then
  echo "Grade: A"
```

```
elif [ $marks -ge 75 ]; then
  echo "Grade: B"
else
  echo "Grade: C"
fi
```

---

4. Nested if Statements

Used when one if is inside another.

Syntax:

```
if [ condition1 ]
then
    if [ condition2 ]
    then
        commands
    fi
fi
```

Example:

```
a=10
b=20
if [ $a -lt 15 ]; then
  if [ $b -gt 15 ]; then
    echo "a < 15 and b > 15"
  fi
fi
```

---

5. case Statement

Checks a variable against multiple patterns.

Syntax:

```
case $variable in
  pattern1)
      commands
      ;;
  pattern2)
      commands
      ;;
  *)
      default_commands
      ;;
esac
```

Example:

```
echo "Enter a number (1-3):"
read num
case $num in
  1) echo "You chose One" ;;
  2) echo "You chose Two" ;;
  3) echo "You chose Three" ;;
  *) echo "Invalid choice" ;;
esac
```

Common Operators:

Arithmetic: -eq, -lt, -gt, -ne

String: =, !=, -z, -n

File Test: -f, -d, -r, -w, -x

Example Combining All:

```
read -p "Enter your age: " age
if [ $age -lt 13 ]; then
  echo "Child"
```

```
elif [ $age -lt 20 ]; then
  echo "Teenager"
elif [ $age -lt 60 ]; then
  echo "Adult"
else
  echo "Senior"
fi
```

Summary Table

Type	Description	Example
if	Runs commands if true	if [$a -gt 10]; then ... fi
if-else	Executes one block if false	if [$x -eq 1]; else ... fi
if-elif-else	Multiple conditions	if ... elif ... else ... fi
Nested if	if inside another if	if [...]; if [...]; fi; fi
case	Pattern-based switch	case $var in ... esac

Loops in Bash (Terminal Style)

Definition:

A loop in Bash repeatedly executes a block of commands until a specific condition is met.

---

1. for Loop

Used to iterate over items or a range of numbers.

Syntax:

```
for variable in list
do
    commands
done
```

Example:

```
for fruit in apple banana cherry
do
  echo "I like $fruit"
done
```

Output:

```
I like apple
I like banana
I like cherry
```

---

2. while Loop

Executes commands as long as the condition is true.

Example:

```
count=1
while [ $count -le 5 ]
```

```
do
  echo "Count = $count"
  ((count++))
done
```

Output:

```
Count = 1
Count = 2
Count = 3
Count = 4
Count = 5
```

---

3. until Loop

Runs commands until the condition becomes true (opposite of while).

Example:

```
num=1
until [ $num -gt 5 ]
do
  echo "Number = $num"
  ((num++))
done
```

---

4. select Loop

Creates a menu for user input.

Example:

```
select drink in Coffee Tea Juice
do
  echo "You selected $drink"
  break
done
```

Output:

```
1) Coffee
2) Tea
3) Juice
#? 2
You selected Tea
```

Arrays in Bash (Terminal Style)

Definition:

An array in Bash is a variable that can hold multiple values at once, accessible using an index number.

---

1. Indexed Arrays

These use numeric indexes (0, 1, 2...).

Example:

```
fruits=("apple" "banana" "cherry")
echo ${fruits[0]}   # apple
echo ${fruits[@]}   # all elements
echo ${#fruits[@]}  # array length
```

Loop Example:

```
for item in "${fruits[@]}"; do
  echo $item
done
```

Output:

```
apple
banana
cherry
```

---

2. Associative Arrays

These use string keys (like dictionaries). Must be declared with declare -A.

Example:

```
declare -A capital
```

```
capital[France]="Paris"
capital[Japan]="Tokyo"
echo ${capital[France]}
```

Loop Example:

```
for country in "${!capital[@]}"; do
  echo "$country -> ${capital[$country]}"
done
```

Output:

```
France -> Paris
Japan -> Tokyo
```

---

3. User Input Arrays

You can take user input and store it in arrays.

Example (Indexed):

```
read -p "Enter three fruits (space separated): " -a fruits
echo "You entered: ${fruits[@]}"
```

Example (Associative):

```
declare -A country
read -p "Enter country: " c
read -p "Enter capital: " cap
country[$c]=$cap
echo "Capital of $c is ${country[$c]}"
```

Output Example:

```
Enter country: Japan
Enter capital: Tokyo
Capital of Japan is Tokyo
```

Summary Table

Type	Declaration	Index Type	Example Access
Indexed Array	Optional	Numeric	${arr[0]}
Associative	declare -A	String	${arr[key]}
User Input	read -a arr	Numeric	${arr[@]}

Exit Status and Error Codes in Bash (Terminal Style)

Definition:

Every command or script in Bash returns an exit status code that indicates whether it succeeded (0) or failed (non-zero).

Example 1: Checking Exit Status

```
#!/bin/bash
ls /home       # Valid command
echo "Exit status: $?"
ls /no_such_dir  # Invalid command
echo "Exit status: $?"
```

Output:

```
Exit status: 0
Exit status: 2
```

Example 2: Using Exit Code in Condition

```
ping -c 1 google.com > /dev/null 2>&1
if [ $? -eq 0 ]; then
  echo "Internet is working"
else
  echo "No internet connection"
fi
```

Example 3: Short Syntax with && and ||

```
ping -c 1 google.com > /dev/null && echo "Online" || echo "Offline"
```

Example 4: Custom Exit Codes

```
#!/bin/bash
if [ $# -lt 1 ]; then
  echo "Error: No argument provided!"
  exit 1
fi
echo "Argument provided: $1"
exit 0
```

Common Exit Codes

Code	Meaning	Description
0	Success	Command executed fine
1	General error	Catch-all for errors
2	Misuse of shell builtins	Syntax or usage error
126	Command cannot execute	Permission denied
127	Command not found	Invalid command
128	Invalid exit argument	Bad exit number
130	Script terminated (Ctrl+C)	Interrupt signal
255	Exit status out of range	Too high return code

Summary Table

Symbol	Description	Example
$?	Last command exit code	echo $?
exit N	Exit script with code N	exit 1
&&	Run next if success	cmd1 && cmd2
||	Run next if failed	cmd1 || cmd2
set -e	Exit script on any error	-

| return N| Return code from function | return 2 |

Redirections and Pipes in Bash - Terminal Style Summary

Definition:

Redirection changes the default input/output flow of commands, while pipes connect output of one command to the input of another, enabling command chaining.

Standard Streams in Bash

stdin (0) - Standard Input (keyboard)

stdout (1) - Standard Output (screen)

stderr (2) - Standard Error (screen)

---

1. Output Redirection (> and >>)

```
command > file      # Overwrite file
command >> file     # Append to file
```

Example:

```
echo "Hello World" > output.txt
ls >> filelist.txt
```

2. Input Redirection (<)

```
command < file
```

Example:

```
wc -l < myfile.txt
```

3. Error Redirection (2>, 2>>)

```
command 2> error.txt  # Redirect errors
command 2>> error.txt # Append errors
```

Example:

```
ls /no_such_dir 2> errors.txt
```

4. Redirect stdout and stderr together

```
command > output.txt 2>&1
```

or

```
command &> output.txt
```

Example:

```
ls /etc /no_dir &> result.txt
```

5. Discard Output (/dev/null)

```
command > /dev/null 2>&1
```

This sends all output and errors to nowhere.

6. Here Document (<<)

```
cat << EOF
This is line one.
This is line two.
EOF
```

7. Here String (<<<)

```
grep "hello" <<< "hello world"
```

8. Combining Redirections

```
ls /etc /no_dir > output.txt 2> error.txt
```

Output and errors stored separately.

PIPES (|)

A pipe connects the output of one command to the input of another.

Syntax:

```
command1 | command2 | command3
```

Examples:

```
ls -l | grep ".sh"          # Show only shell scripts
ps aux | wc -l              # Count processes
```

```
cat /etc/passwd | sort      # Sort users
cat names.txt | sort | uniq # Remove duplicates
cat /var/log/syslog | grep "error" > errors.txt
```

Quick Reference Table

Operator	Description	Example
>	Redirect output (overwrite)	echo hi > file.txt
>>	Redirect output (append)	echo hi >> file.txt
<	Redirect input	wc -l < file.txt
2>	Redirect errors	ls /no_dir 2> err.txt
&>	Redirect all output	cmd &> all.txt
|	Pipe output	ls | grep txt
<<	Here Document	cat << EOF
<<<	Here String	grep word <<< "text"

Debugging Commands in Bash (Terminal Style)

Definition:

Debugging in Bash helps identify and fix errors by tracing, printing, and controlling script execution.

Common Debugging Options

Option	Description
set -x	Print each command before executing
set +x	Stop printing commands
bash -x script.sh	Run script in debug mode
set -e	Exit immediately if command fails
set -v	Print lines before executing
trap ERR	Run command when an error occurs
echo	Print variable values manually

Example 1: Using set -x and set +x

```
#!/bin/bash
echo "Start script"
set -x
ls /home
pwd
set +x
echo "End script"
```

Example 2: Running script with bash -x

```
bash -x myscript.sh
```

```
-----------------------------------------
Example 3: set -e (Exit on Error)
#!/bin/bash
set -e
echo "Start"
ls /no_dir     # This fails
echo "End"     # Will not execute

-----------------------------------------
Example 4: set -u (Unset Variables)
#!/bin/bash
set -u
echo $name

-----------------------------------------
Example 5: Using trap to Handle Errors
#!/bin/bash
trap 'echo "Error on line $LINENO"' ERR
echo "Running command..."
ls /no_such_dir
echo "This will not run"

-----------------------------------------
Example 6: Customize Debug Output with PS4
PS4='+ [${BASH_SOURCE}:${LINENO}] '
set -x
echo "Testing script"
ls /etc

-----------------------------------------
Example 7: Syntax Check Only (-n)
bash -n script.sh
```

Summary Table

Option	Description	Example
set -x	Print each command	set -x
set +x	Stop debug printing	set +x
bash -x	Run script in debug mode	bash -x script.sh
set -e	Exit on first error	set -e
set -u	Error on unset variable	set -u
trap ERR	Run command on error	trap 'cmd' ERR
PS4	Customize debug prefix	PS4='+ [${LINENO}] '
bash -n	Syntax check only	bash -n script.sh

File and Directory Handling Commands in Bash (Terminal Style)

Definition:

File and directory handling commands in Bash are used to create, view, modify, move, copy, delete, and manage files and folders in Linux systems.

---

1. File Handling Commands

```
cat     - Display contents of a file
tac     - Display file in reverse order
nl      - Display file with line numbers
head    - Show first lines of file
tail    - Show last lines of file
touch   - Create new empty file
cp      - Copy a file
mv      - Move or rename a file
rm      - Delete file
```

Examples:

```
cat notes.txt
nl notes.txt
head -5 notes.txt
tail -n 3 notes.txt
```

---

2. Directory Handling Commands

```
pwd     - Show current working directory
ls      - List directory contents
cd      - Change directory
mkdir   - Create a new directory
rmdir   - Remove empty directory
rm -r   - Remove directory and contents
```

```
cp -r   - Copy directory recursively
mv      - Move or rename a directory

Examples:
pwd
ls -lh
cd /etc
mkdir projects
rm -r old_folder

-----------------------------------------
3. File Search and Filtering
find    - Search for files or directories
locate  - Quickly find files (uses database)
grep    - Search for text patterns
wc      - Count lines, words, or bytes
sort    - Sort lines alphabetically
uniq    - Remove duplicate lines
cut     - Extract sections from each line

Examples:
find /etc -name '*.conf'
grep 'bash' /etc/passwd
wc -l file.txt
sort names.txt | uniq

-----------------------------------------
4. File Compression and Archiving
tar     - Archive files and directories
gzip    - Compress files
gunzip  - Decompress files
zip     - Create zip archive
unzip   - Extract zip archive
```

Examples:

```
tar -cvf backup.tar folder/
tar -xvf backup.tar
gzip notes.txt
gunzip notes.txt.gz
zip myzip file1 file2
unzip myzip.zip
```

---

5. File Permission and Ownership Commands

```
chmod   - Change file permissions
chown   - Change file owner
chgrp   - Change group ownership
umask   - Set default permission mask
```

Examples:

```
chmod +x script.sh
chmod 644 file.txt
chown user:user file.txt
chgrp developers file.txt
```

Summary Table

Category	Common Commands
File Ops	cat, nl, head, tail, cp, mv, rm, touch
Dir Ops	pwd, ls, cd, mkdir, rmdir, cp -r, rm -r
Search	find, locate, grep, wc, sort, uniq, cut
Archive	tar, gzip, zip, unzip, gunzip
Perms	chmod, chown, chgrp, umask

Text Processing Commands in Bash (Terminal Style)

Definition:

Text processing commands in Bash are used to manipulate and analyze text files or streams. They can search, sort, filter, replace, and transform data efficiently.

---

1. Common Text Processing Commands

```
cat         - Display file contents
grep        - Search for patterns in text
sort        - Sort lines alphabetically or numerically
uniq        - Remove duplicate lines
cut         - Extract columns or fields from a line
tr          - Translate or delete characters
awk         - Process text using patterns and actions
sed         - Stream editor for text substitution
wc          - Count words, lines, or characters
head        - Show first N lines
tail        - Show last N lines
paste       - Merge lines from multiple files
join        - Join two files on a common field
```

cat - Display and Combine Files

```
cat file.txt
cat -n file.txt  # Show line numbers
```

grep - Search for Text

```
grep 'error' file.txt
grep -i 'bash' file.txt
grep -v 'fail' log.txt
```

```
sort - Sort Lines
sort names.txt
sort -r names.txt
sort -u names.txt

uniq - Remove Duplicates
uniq list.txt
uniq -c list.txt
sort list.txt | uniq -c

cut - Extract Fields
cut -d',' -f1 file.csv
cut -d':' -f1,3 /etc/passwd
cut -c1-10 file.txt

tr - Translate/Delete Characters
tr 'a-z' 'A-Z' < file.txt
tr -d '0-9' < file.txt

awk - Pattern-based Processing
awk '{print $1}' file.txt
awk '/bash/ {print $1, $3}' /etc/passwd

sed - Stream Editing
sed 's/error/issue/g' log.txt
sed '/^$/d' file.txt
sed -n '1,5p' file.txt

wc - Word Count
wc -l file.txt
wc -w file.txt
wc -c file.txt
```

paste/join - Merge Files

```
paste file1.txt file2.txt
join names.txt marks.txt
```

Summary Table

Command	Description	Example
cat	Display file contents	cat file.txt
grep	Search text patterns	grep 'error' log.txt
sort	Sort lines	sort -r file.txt
uniq	Remove duplicates	uniq list.txt
cut	Extract columns	cut -d',' -f2 file.csv
tr	Translate characters	tr 'a-z' 'A-Z'
awk	Process text	awk '{print $1}' file.txt
sed	Stream edit text	sed 's/old/new/g' file.txt
wc	Count words/lines	wc -l file.txt
paste	Merge files	paste file1 file2
join	Join files	join f1 f2

Cron Jobs Commands in Bash (Terminal Style)

Definition:

A cron job is a time-based scheduler in Linux used to run commands or scripts automatically at specified times or intervals.

Crontab Syntax:

```
* * * * * command_to_execute
| | | | |
| | | | +-- Day of week (0-6, Sunday=0)
| | | +---- Month (1-12)
| | +------ Day of month (1-31)
| +-------- Hour (0-23)
+---------- Minute (0-59)
```

Examples:

```
30 2 * * * /home/user/backup.sh  # Runs daily at 2:30 AM
*/5 * * * * /home/user/ping.sh   # Every 5 minutes
0 0 * * 0 /home/user/clean.sh    # Every Sunday
```

Cron Commands:

```
crontab -e                - Edit current user's crontab
crontab -l                - List current user's cron jobs
crontab -r                - Remove current user's cron jobs
crontab -u user -e        - Edit another user's crontab (root only)
service cron start        - Start cron service
service cron status       - Check cron service status
systemctl restart cron    - Restart cron service
```

Common Scheduling Examples:

```
* * * * *       - Every minute
*/5 * * * *     - Every 5 minutes
0 * * * *       - Every hour
0 0 * * *       - Every day at midnight
0 0 * * 0       - Every Sunday
0 0 1 * *       - Every 1st of month
0 9 * * 1-5     - Every weekday at 9 AM
```

Redirecting Output:

```
0 2 * * * /home/user/script.sh >> /home/user/logs.txt 2>&1
```

Saves both output and errors to logs.txt

View Cron Logs:

```
grep CRON /var/log/syslog
journalctl -u cron
```

Special Cron Keywords:

```
@reboot    - Run once at startup
@yearly    - Run once a year
@monthly   - Run once a month
@weekly    - Run once a week
@daily     - Run once a day
@hourly    - Run once every hour
```

Examples with Keywords:

```
@reboot /home/user/startup.sh
@daily /home/user/cleanup.sh
@weekly /home/user/backup.sh
```

System vs User Cron Jobs:

User Cron: crontab -e

System Cron: /etc/crontab

Cron Folders: /etc/cron.daily, /etc/cron.hourly

Example /etc/crontab:

```
SHELL=/bin/bash
PATH=/sbin:/bin:/usr/sbin:/usr/bin
0 0 * * * root /usr/bin/apt update -y
```

Summary Table:

crontab -e	- Edit cron jobs
crontab -l	- List cron jobs
*/5 * * * *	- Run every 5 minutes
@daily	- Run every day
@reboot	- Run at startup
service cron status	- Check cron status
journalctl -u cron	- View cron logs

Script Arguments in Bash (Terminal Style)

Definition:

Script arguments are values passed to a Bash script when it is executed. They make scripts more dynamic and reusable.

Accessing Script Arguments

Special variables let you access command-line arguments easily.

Common Variables:

```
$0  -> Script name
$1  -> First argument
$2  -> Second argument
$#  -> Number of arguments
$@  -> All arguments separately
$*  -> All arguments as one string
$?  -> Exit status of last command
$$  -> PID of current script
```

Example 1: Simple Argument Access

```
#!/bin/bash
echo "Script name: $0"
echo "First argument: $1"
echo "Second argument: $2"
echo "Total arguments: $#"
```

Run:

```
./script.sh apple banana
```

Output:

```
Script name: ./script.sh
```

```
First argument: apple
Second argument: banana
Total arguments: 2

-----------------------------------------
Example 2: Loop Through Arguments
#!/bin/bash
echo "You passed $# arguments."
for arg in "$@"
do
  echo "Argument: $arg"
done

Output:
You passed 3 arguments.
Argument: 10
Argument: 20
Argument: 30

-----------------------------------------
Example 3: Using Arguments in Calculations
#!/bin/bash
sum=$(( $1 + $2 ))
echo "Sum: $sum"

Run:
./script.sh 5 7

Output:
Sum: 12

-----------------------------------------
Example 4: Handling Missing Arguments
#!/bin/bash
```

```
if [ $# -lt 2 ]; then
  echo "Usage: $0 num1 num2"
  exit 1
fi
echo "You entered $1 and $2"
```

Output:

```
Usage: ./script.sh num1 num2
```

Example 5: Using shift Command

```
#!/bin/bash
echo "All arguments: $@"
shift
echo "After shift: $@"
```

Run:

```
./script.sh a b c
```

Output:

```
All arguments: a b c
After shift: b c
```

Summary Table

Variable	Description	Example
$0	Script name	./myscript.sh
$1, $2...	Positional arguments	$1 -> first arg
$#	Number of arguments	3
$@	All arguments (separately)	arg1 arg2 arg3
$*	All arguments (single str)	"arg1 arg2 arg3"
$$	PID of script	12345

| $? | Exit status | 0 (success) |
| $! | PID of last bg process | - |

Comments and Functions in Bash (Terminal Style)

Definition:

Comments are lines ignored by the shell, used to document code, explain logic, or disable parts of a script.

Types of Comments:

```
# This is a comment                - Single-line comment
echo 'Hello' # Inline comment      - Inline comment
: ' ... '                          - Multi-line comment using colon and quotes
<<COMMENT ... COMMENT              - Multi-line comment using here-doc
```

Examples:

```
# This script prints system info
echo 'System Information:'
uname -a

: '
Multi-line comment block.
Used to describe code sections.
'
```

Functions in Bash:

Functions are reusable code blocks that perform specific tasks and help avoid repetition.

Syntax:

```
function function_name { commands }
```

or

```
function_name() { commands }

Example 1: Basic Function
greet() {
    echo 'Hello, $1!'
}
greet 'Alice'
# Output: Hello, Alice!

Example 2: Using Local Variables
calculate_sum() {
    local a=$1
    local b=$2
    echo 'Sum: $((a + b))'
}
calculate_sum 5 10
# Output: Sum: 15

Example 3: Return Value
add() { echo $(( $1 + $2 )); }
result=$(add 4 6)
echo 'Result: $result'

Example 4: Function Calling Another
say_hello() { echo 'Hello!'; }
say_goodbye() { say_hello; echo 'Goodbye!'; }
say_goodbye

Example 5: Function with Condition
check_number() {
    if [ $1 -gt 0 ]; then
        echo 'Positive'
    elif [ $1 -lt 0 ]; then
```

```
        echo 'Negative'
    else
        echo 'Zero'
    fi
}

Example 6: Exit Status
is_even() {
    if (( $1 % 2 == 0 )); then return 0; else return 1; fi
}
is_even 4 && echo 'Even' || echo 'Odd'

Example 7: Recursive Function
factorial() {
    if [ $1 -le 1 ]; then echo 1; else local prev=$(factorial $(( $1 - 1
))); echo $(( $1 * prev )); fi
}
result=$(factorial 5)
echo 'Factorial: $result'

-----------------------------------------
Summary Table:
# comment           - Single-line comment
: '...'             - Multi-line comment
function_name()     - Define a function
local var=value     - Create local variable
return N            - Exit with status code
$?                  - Last command exit status
$(command)          - Command substitution
```